Step 8 - Relative vs Absolute Time (TVAL vs CVAL) ..... 1
Reference Note ..... 1
Learning Objectives ..... 1
1. One Timer, Two Programming Models ..... 1
2. Relative Time (TVAL) ..... 2
Hardware Operation ..... 3
3. Absolute Time (CVAL) ..... 3
4. Relative vs Absolute Comparison ..... 4
5. Example Using a 24 MHz Counter ..... 4
Relative ..... 4
Absolute ..... 5
6. Periodic Events ..... 5
Approach 1 - Relative Scheduling ..... 5
7. Scheduler Drift ..... 6
Next Interrupt ..... 7
8. Absolute Scheduling ..... 7
Why It Works ..... 7
9. Why Modern Systems Prefer CVAL ..... 8
Architecture View ..... 8
Common Misconceptions ..... 8
Misconception 1 ..... 9
Misconception 2 ..... 9
Misconception 3 ..... 9
Key Takeaways ..... 9
Knowledge Check ..... 10
Q1 ..... 10
Q2 ..... 10
Q3 ..... 10

Q4 ..... 10
Q5 ..... 10
Answer Key ..... 10

# Step 8 - Relative vs Absolute Time (TVAL vs CVAL)

## Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture documentation.

The ARM Generic Timer provides two ways to program the same timer:

- **Relative Time** using TVAL
- **Absolute Time** using CVAL

Understanding this distinction is critical because it directly impacts timer accuracy, scheduler design, and drift behavior.

# Learning Objectives

After completing this chapter, you should understand:

- Difference between Relative and Absolute Time
- Purpose of TVAL
- Purpose of CVAL
- How hardware handles TVAL writes
- Why periodic timers prefer absolute deadlines
- What scheduler drift is
- Why modern systems primarily use CVAL

# 1. One Timer, Two Programming Models

A very common misconception is:

TVAL = One Timer
CVAL = Another Timer

This is incorrect.

| Register | Purpose |
|---|---|
| CNTP_TVAL_EL0 | Relative Programming |
| CNTP_CVAL_EL0 | Absolute Programming |

Both program the **same Physical Timer**.

Likewise:

| Register | Purpose |
|---|---|
| CNTV_TVAL_EL0 | Relative Programming |
| CNTV_CVAL_EL0 | Absolute Programming |

Both program the **same Virtual Timer**.

# 2. Relative Time (TVAL)

Relative time means:

Fire after X ticks from now

Example:

| Current Counter | 1,000,000 |
|---|---|
| TVAL | 24,000 |

Meaning:

Generate event
24,000 ticks from now

### Hardware Operation

When software writes:

CNTP_TVAL_EL0 = 24,000

hardware internally performs:

CNTP_CVAL
=
CurrentCounter + 24,000

If:

CurrentCounter = 1,000,000

then:

CNTP_CVAL
=
1,024,000

This translation is done automatically.

## 3. Absolute Time (CVAL)

Absolute time means:

Fire when counter reaches Y

Example:

| Current Counter | 1,000,000 |
| --- | --- |
| CVAL | 1,024,000 |

Meaning:

Generate event when
Counter = 1,024,000

The timer continuously compares:

Current Count
vs
Target Count

and expires when:

Current Count >= CVAL

# 4. Relative vs Absolute Comparison

| Feature | TVAL | CVAL |
|---|---|---|
| Programming Model | Relative | Absolute |
| Meaning | Delay from now | Target count |
| Hardware Uses CVAL Internally | Yes | Yes |
| Easier to Understand | Yes | Moderate |
| Best for Periodic Timing | No | Yes |

# 5. Example Using a 24 MHz Counter

### Relative

| Parameter | Value |
|---|---|
| Counter | 1,000,000 |
| TVAL | 24,000 |

Result:

Event after 1 ms

Hardware calculates:

CVAL = 1,024,000

### Absolute

Program directly:

| Parameter | Value |
|---|---|
| Counter | 1,000,000 |
| CVAL | 1,024,000 |

Result:

Event at count 1,024,000

Same timer event.

Set CVAL again for next target time

Different programming model.

# 6. Periodic Events

Suppose we need:

Interrupt every 1 ms

Period:

24,000 ticks

## Approach 1 - Relative Scheduling

After every interrupt:

New Deadline

=
Current Counter + 24,000

Example:

| Interrupt | Actual Counter |
|---|---|
| #1 | 1,024,200 |
| #2 | 1,048,350 |
| #3 | 1,072,600 |

Small delays begin accumulating.

# 7. Scheduler Drift

Suppose timer should expire at:

1,024,000

ISR actually starts at:

1,024,200

Latency:

200 ticks

If software programs:

CurrentCounter + Period

then:

1,024,200 + 24,000
=
1,048,200

Expected:

1,048,000

Error:

200 ticks

### Next Interrupt

Suppose another:

100 tick delay

Now:

Error = 300 ticks

This accumulated timing error is called:

Scheduler Drift

# 8. Absolute Scheduling

Instead:

Deadline += Period

Example:

| Event | Deadline |
|---|---|
| 1 | 1,024,000 |
| 2 | 1,048,000 |
| 3 | 1,072,000 |
| 4 | 1,096,000 |

Even if the ISR executes late:

Schedule remains aligned.

# Why It Works

Interrupt latency affects:

When ISR executes

but not:

Where deadlines are placed

This prevents accumulated drift.

# 9. Why Modern Systems Prefer CVAL

| Requirement | Preferred Register |
|---|---|
| One-shot delay | TVAL or CVAL |
| Periodic timer | CVAL |
| Accurate scheduling | CVAL |
| Long-term stability | CVAL |

General recommendation:

TVAL = Convenience
CVAL = Precision

# Architecture View

| Register | Purpose |
|---|---|
| CNTPCT_EL0 | Current counter |
| CNTP_TVAL_EL0 | Relative interval |
| CNTP_CVAL_EL0 | Absolute deadline |
| CNTP_CTL_EL0 | Timer control |

# Common Misconceptions

## Misconception 1

TVAL and CVAL are different timers

False.

They program the same timer.

## Misconception 2

TVAL is more accurate

False.

CVAL generally provides better long-term periodic timing.

## Misconception 3

Timer automatically reloads

False.

Software typically programs the next deadline.

# Key Takeaways

| Concept | Key Point |
| --- | --- |
| TVAL | Relative time |
| CVAL | Absolute time |
| Timer Hardware | Uses compare value |

| | |
|---|---|
| Drift | Caused by relative scheduling |
| Absolute Deadlines | Avoid accumulated drift |
| Recommended For Periodic Events | CVAL |

# Knowledge Check

### Q1

What is the difference between TVAL and CVAL?

### Q2

Are TVAL and CVAL different timers?

### Q3

What happens internally when TVAL is written?

### Q4

What causes scheduler drift?

### Q5

Why do periodic systems typically prefer CVAL?

# Answer Key

| Question | Answer |
|---|---|
| Q1 | TVAL is relative; CVAL is absolute |
| Q2 | No, they program the same timer |
| Q3 | Hardware computes CVAL = CurrentCounter + TVAL |
| Q4 | Accumulated interrupt latency in relative scheduling |
| Q5 | Absolute deadlines prevent accumulated drift |